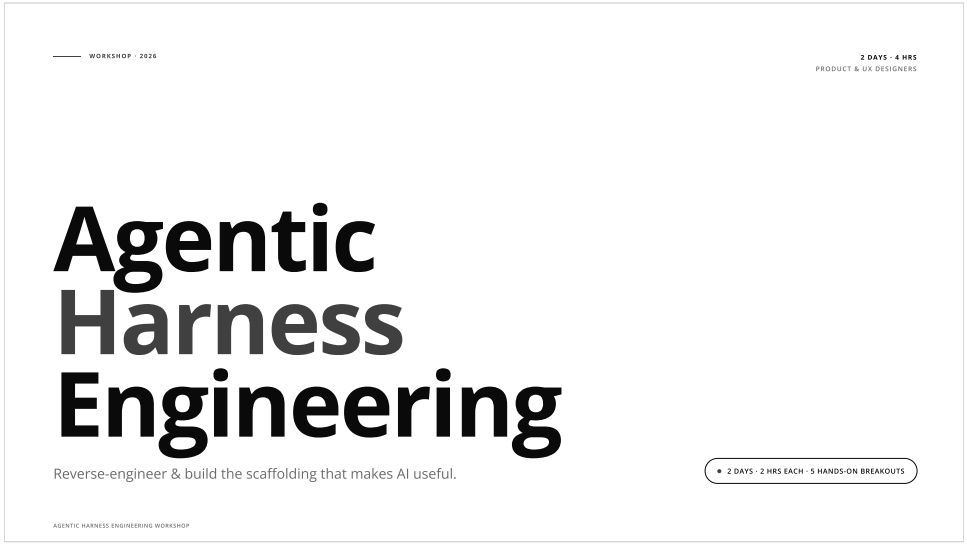
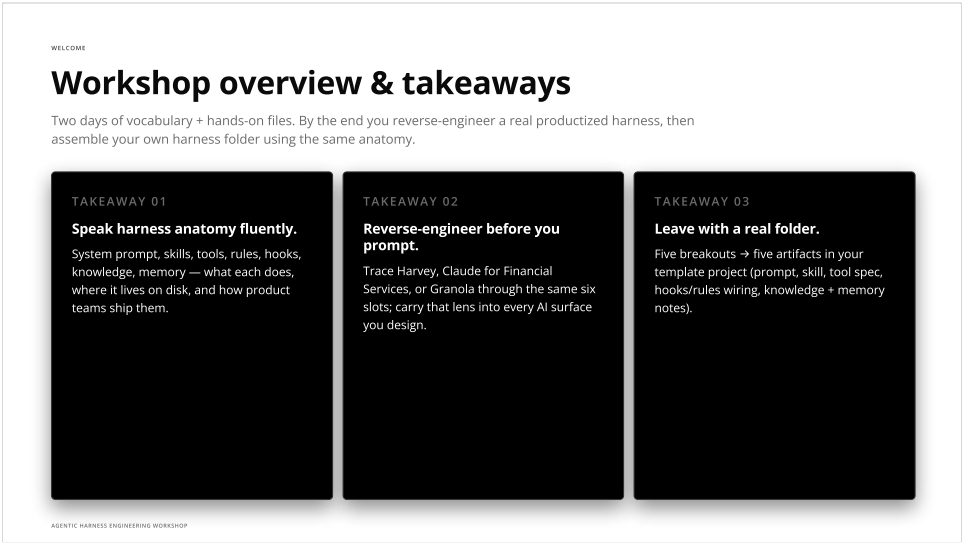


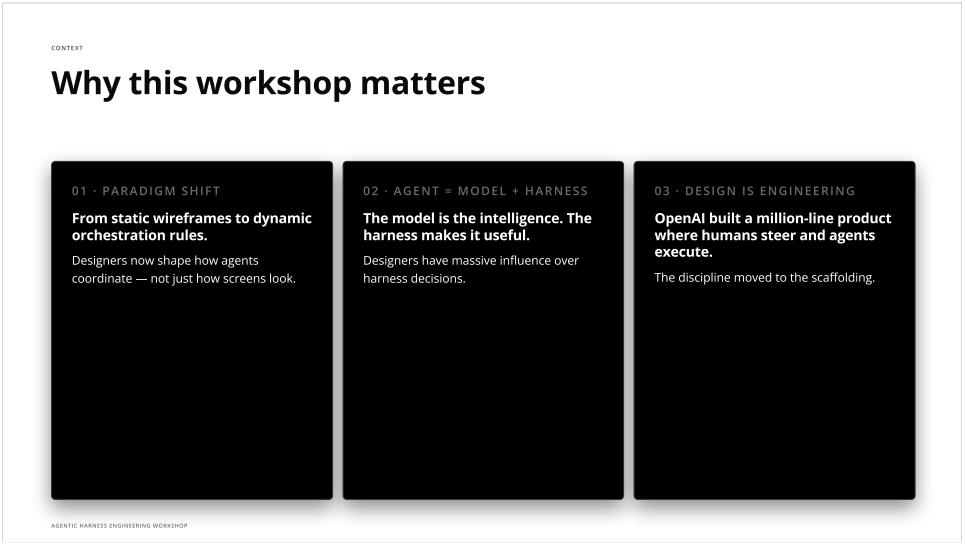
SPEAKER NOTES

Welcome to Agentic Harness Engineering for Product & UX Designers. This is a 2-day, 2-hour-per-day workshop. Quick show of hands — who's been using RooCode? Copilot? Both? Great. You already touch the harness every day — you just call it 'rules' or 'instructions'. Today and tomorrow we name what you're already doing, give it a real anatomy, and you build one from scratch by reverse-engineering an AI app you actually use. By the end of Day 2 you'll have a runnable harness folder you can drop into any project.



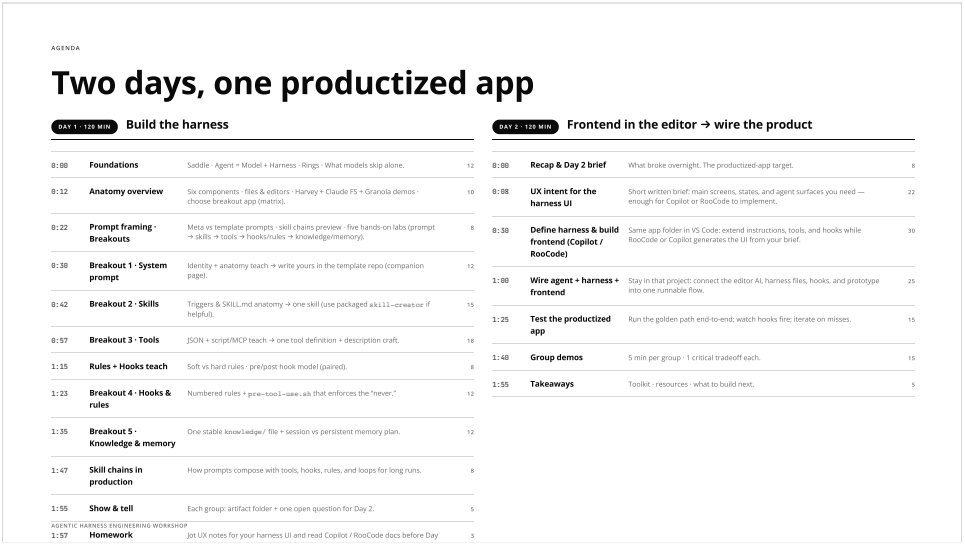
SPEAKER NOTES

Workshop overview & takeaways (slide 1b). Give ~30 seconds per card: fluent anatomy vocabulary, reverse-engineer before raw prompting, leave with artifacts in the template repo. Tie each takeaway to today's five breakouts without opening the facilitator manual.



SPEAKER NOTES

The role of design is shifting from crafting interfaces to orchestrating intelligent systems. CARD 1 — Paradigm shift: from static wireframes to dynamic orchestration rules. CARD 2 — Agent = Model + Harness. The model is the brain. The harness is the body, the workshop, and the rules of engagement. Designers shape the harness. CARD 3 — Design IS engineering. OpenAI shipped a million-line product where humans steered and Codex executed. The discipline is now in the scaffolding.



AGENDA

Two days, one productized app

DAY 1 · 120 MIN **Build the harness**

0:00	**Foundations**	Saddle · Agent = Model + Harness · Rings · What models skip alone.	12
0:12	**Anatomy overview**	Six components · files & editors · Harvey + Claude FS + Granola demos · choose breakout app (matrix).	10
0:22	**Prompt framing · Breakouts**	Meta vs template prompts · skill chains preview · five hands-on labs (prompt → skills → tools → hooks/rules → knowledge/memory).	8
0:30	**Breakout 1 · System prompt**	Identity + anatomy teach → write yours in the template repo (companion page).	12
0:42	**Breakout 2 · Skills**	Triggers & SKILL.md anatomy → one skill (use packaged `skill-creator` if helpful).	15
0:57	**Breakout 3 · Tools**	JSON + script/MCP teach → one tool definition + description craft.	18
1:15	**Rules + Hooks teach**	Soft vs hard rules · pre/post hook model (paired).	8
1:23	**Breakout 4 · Hooks & rules**	Numbered rules + `pre-tool-use.sh` that enforces the "never."	12
1:35	**Breakout 5 · Knowledge & memory**	One stable `knowledge/` file + session vs persistent memory plan.	12
1:47	**Skill chains in production**	How prompts compose with tools, hooks, rules, and loops for long runs.	8
1:55	**Show & tell**	Each group: artifact folder + one open question for Day 2.	5
1:57	**Homework**	Jot UX notes for your harness UI and read Copilot / RooCode docs before Day	3

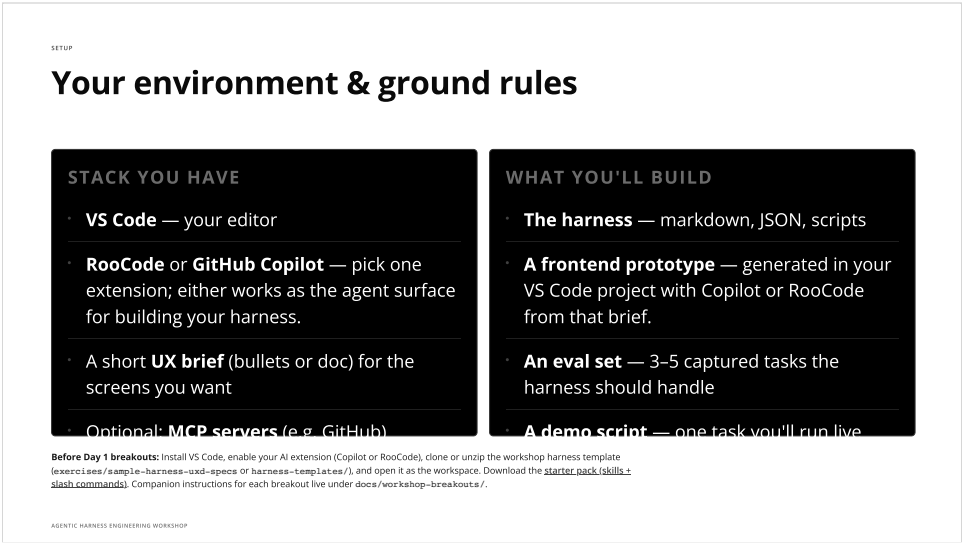
DAY 2 · 120 MIN **Frontend in the editor → wire the product**

0:00	**Recap & Day 2 brief**	What broke overnight. The productized-app target.	8
0:08	**UX intent for the harness UI**	Short written brief: main screens, states, and agent surfaces you need — enough for Copilot or RooCode to implement.	22
0:30	**Define harness & build frontend (Copilot / RooCode)**	Same app folder in VS Code: extend instructions, tools, and hooks while RooCode or Copilot generates the UI from your brief.	30
1:00	**Wire agent + harness + frontend**	Stay in that project: connect the editor AI, harness files, hooks, and prototype into one runnable flow.	25
1:25	**Test the productized app**	Run the golden path end-to-end; watch hooks fire; iterate on misses.	15
1:40	**Group demos**	5 min per group · 1 critical tradeoff each.	15
1:55	**Takeaways**	Toolkit · resources · what to build next.	5

AGENTIC HARNESS ENGINEERING WORKSHOP

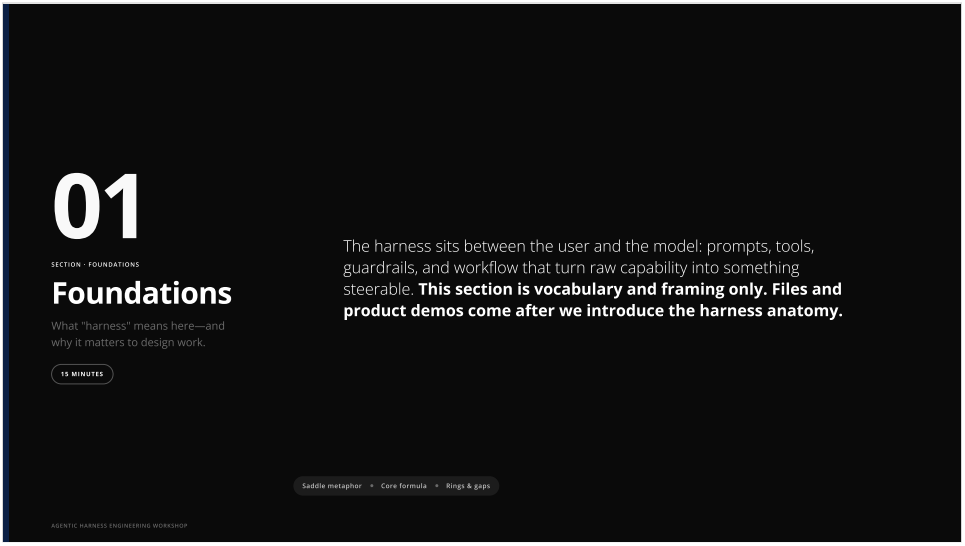
SPEAKER NOTES

Day 1 you build the harness for your product. Day 2 you write a short UX brief, then in one VS Code app project use GitHub Copilot or RooCode to refine the harness and build the UI before wiring agent, harness, and UX together. — Two days, one product. Day 1 you build the harness for your chosen app — six components, taught and worked in focused breakouts. Day 2 changes shape: you clarify the frontend in prose or light sketches, generate the code with Copilot or RooCode, then wire the agent + harness + frontend UX together into a productized demo. I want you 60% hands-on, 40% with me. Let's go.



SPEAKER NOTES

We're not training models or running servers today. You'll author the harness and a frontend prototype, then let your editor's AI play the agent. GROUND RULES — read this slide carefully because it shapes everything. CRITICAL CONSTRAINT: API keys are only available INSIDE the RooCode and Copilot extensions today. You cannot build a standalone agent that calls a model directly. The extensions already have the key — you don't. So we don't pretend to ship a real backend. Instead, we work with what you DO have: VS Code + RooCode + Copilot. THE EDITOR IS THE RUNTIME. You author two things: (1) the harness — markdown, JSON, scripts in a folder — and (2) a frontend prototype built with Copilot or RooCode from your UX brief. Then you load both into VS Code and let the extension's AI play the harnessed agent. Your CLAUDE.md is its system prompt. Your hooks fire on real tool calls. Your prototype is what a user clicks. This is exactly what Day 2's wire-up exercise asks you to do. One sentence to remember: 'harness + frontend = product; the editor's AI is the runtime.'

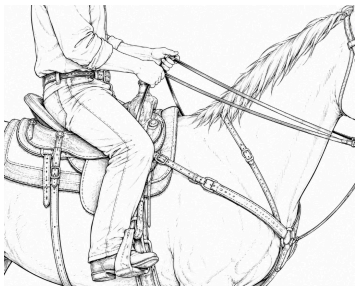


SPEAKER NOTES

[Pause.] Foundations: name what sits between user and model—then we'll mirror it into files and products.

FOUNDATIONS · THE METAPHOR

What is a harness?



THE HORSE

The model has the power.

Raw intelligence — fast, capable, untrained on your task. Powerful enough to throw a rider.

THE SADDLE

The harness makes it ridable.

A seat to sit on, stirrups to steer, a girth to hold the work in place. Not the power — the leverage.

THE RIDER

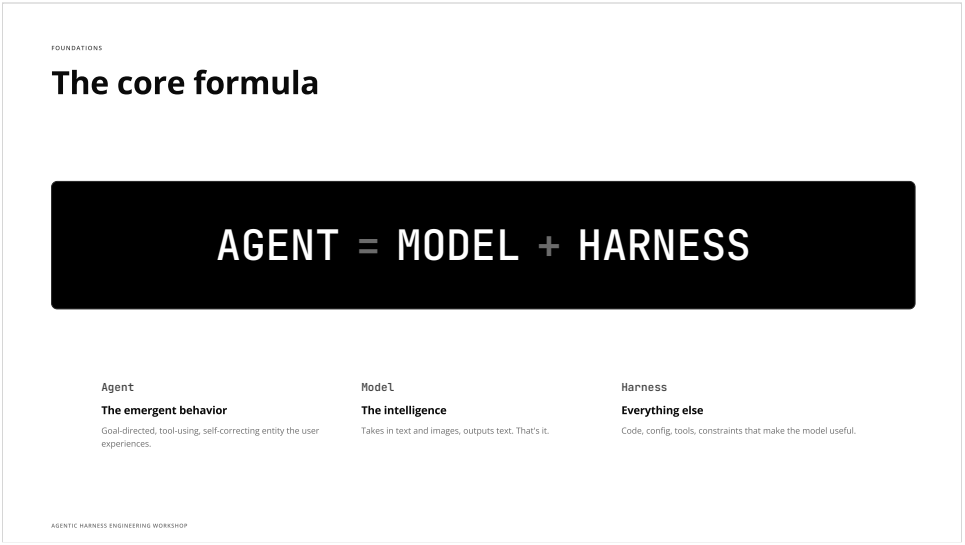
The user gives direction.

Without the saddle, the user is on the ground. With it, they go where they meant to go.

AGENTIC HARNESS ENGINEERING WORKSHOP

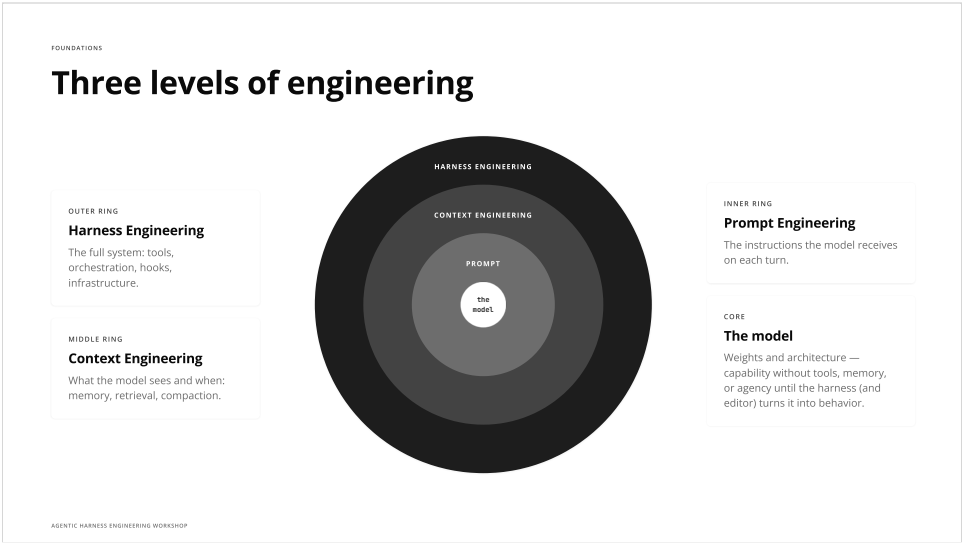
SPEAKER NOTES

A model is a brilliant mind with no body. The harness is the tack that lets it carry weight, take direction, and do useful work. — Plant the metaphor. The model is the horse — fast, strong, but it has no reins, no seat, no destination. The saddle isn't the power; it's the leverage. Three pieces map across: model = horse (raw capability), harness = saddle (tools, rules, memory, hooks — the tack that makes the horse ridable), user = rider (gives direction). This is the working metaphor for the rest of Day 1: every component you build is a piece of saddle. If a model alone could do it, you wouldn't be here.



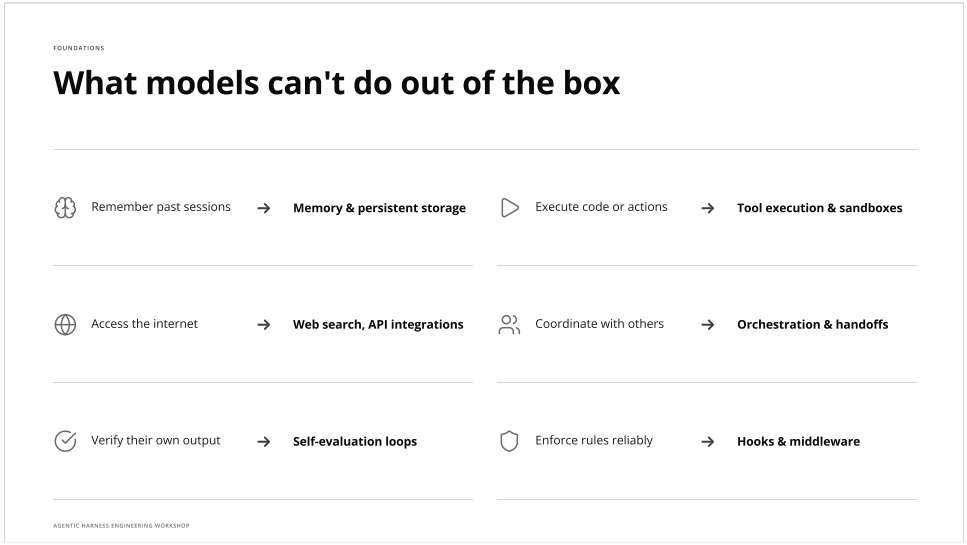
SPEAKER NOTES

Most important idea in the workshop. Agent = Model + Harness. Vivek Trivedy at LangChain: 'If you're not the model, you're the harness.' The AGENT is what the user experiences. The MODEL takes text and images and emits text — that's all. The HARNESS is everything else: code, config, files, tools, constraints. Designers don't train models — they absolutely design harnesses.



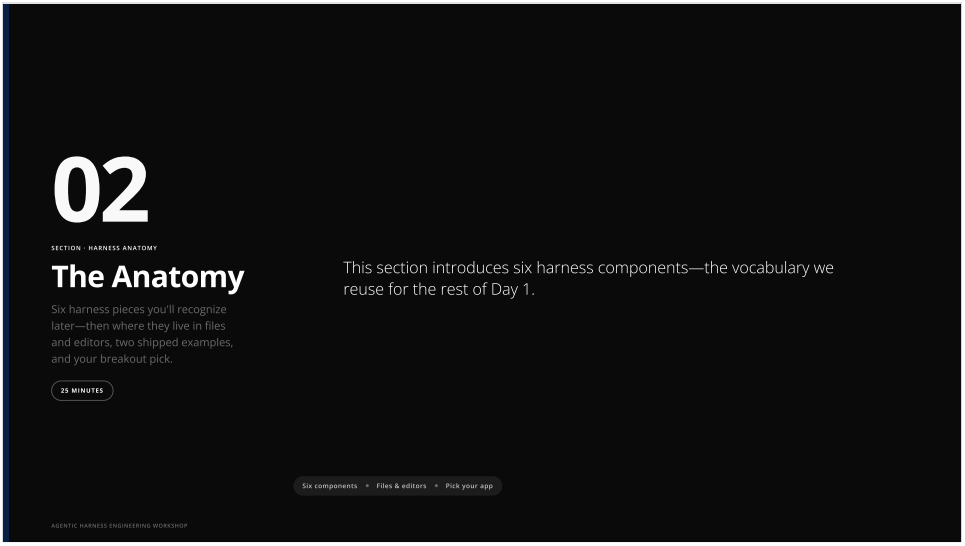
SPEAKER NOTES

Three concentric levels. Prompt — what the model sees this turn. Context — what the model sees and when across turns. Harness — the entire system. We touch all three but harness is where product decisions live.



SPEAKER NOTES

Every limitation is solved by a harness feature. This is where your design decisions live. — Six things models can't do alone. Each one solved by a harness feature. Memory · tool execution · web · coordination · self-verification · enforcement. Every row is a design decision your team will make.



SPEAKER NOTES

SECTION divider · The Anatomy

[Pause.]

We're opening section two.

Foundations ended with what models can't do alone—and how harness pieces answer each gap.

Now everyone gets the same map before we go deep: shared vocabulary first, then the marathon teach-and-dive runs.

I'll walk through what's coming. Say this slowly—the slide is the cheat sheet.

First: the six-slot overview on one screen. Orient the room; don't ask anyone to memorize yet.

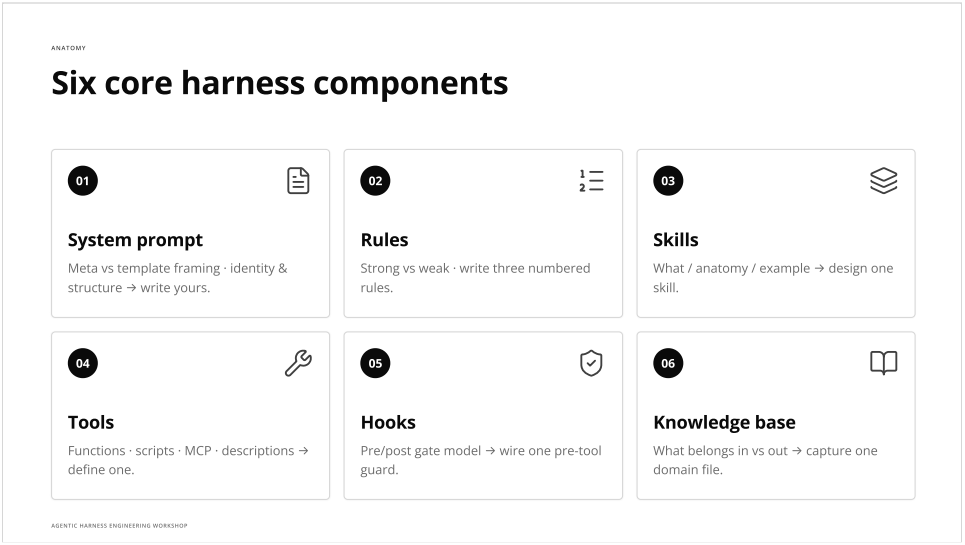
Next: pairwise slides and the leverage map—we widen the picture.

Then: the file-tree slide—where those slots land as real folders.

Then: two shipped demos—Harvey and Claude for Financial Services. Same six labels; different products.

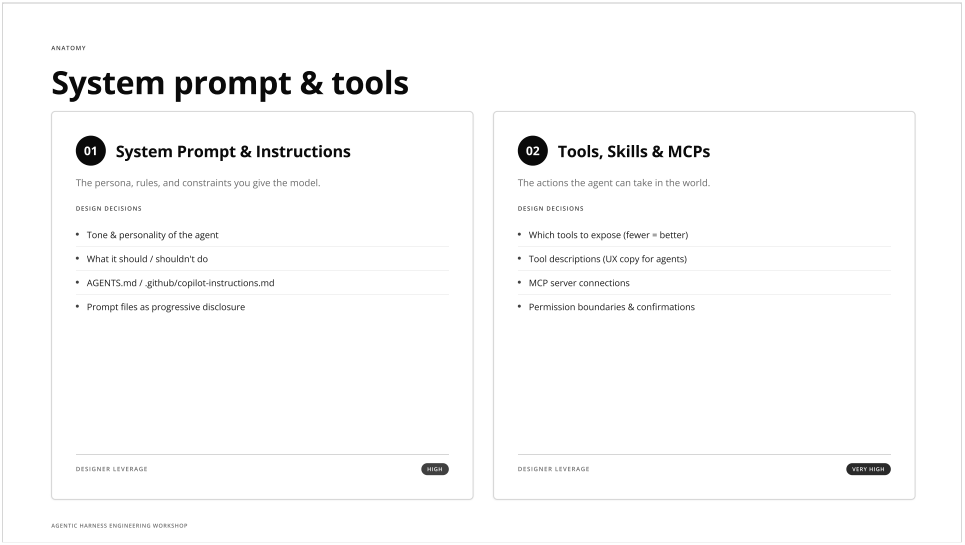
Last in this block: you pick one breakout app from the matrix—Harvey, Claude Financial Services, or Granola.

Close with this line: the slow, component-by-component pass starts in section three, right after you've committed to that pick.



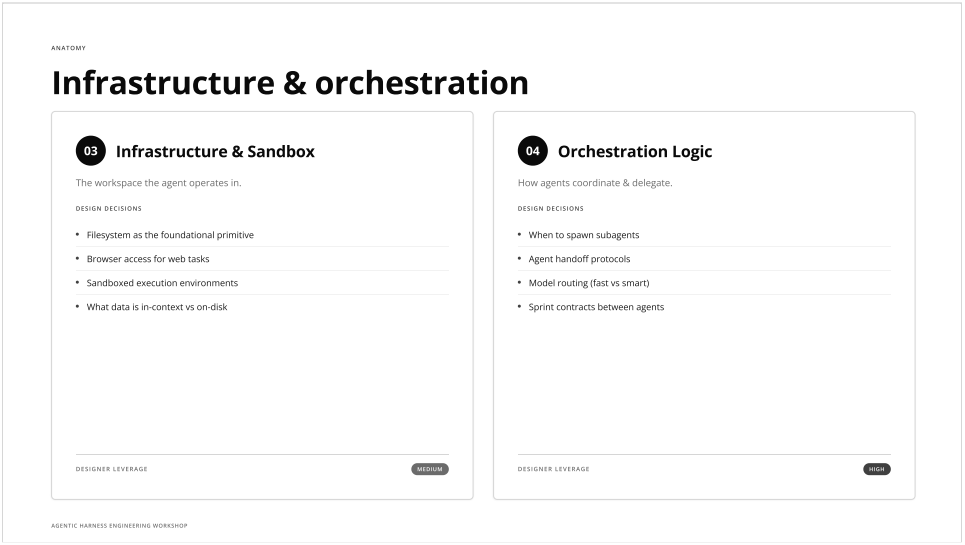
SPEAKER NOTES

Six cards mirror Day 1 agenda breakouts after anatomy—not abstract buckets: system prompt, rules, skills, tools, hooks, knowledge base. Prompt-structure framing folds into card 01; memory is teach-only later (see agenda). Say explicitly each chip maps to a breakout you'll run.



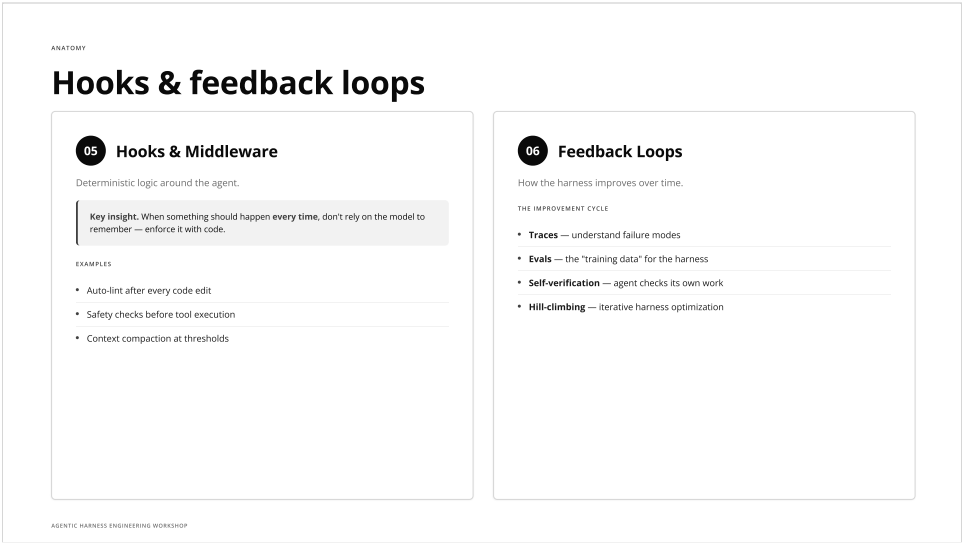
SPEAKER NOTES

LEFT: System prompt = persona, rules, constraints. AGENTS.md and .github/copilot-instructions.md tell the agent how to behave. Prompt files are progressive disclosure. RIGHT: Tools are the actions in the world. Insight: too many MCP servers make agents dumb. Tool descriptions are UX copy for the model — fewer, sharper tools beat a kitchen sink.



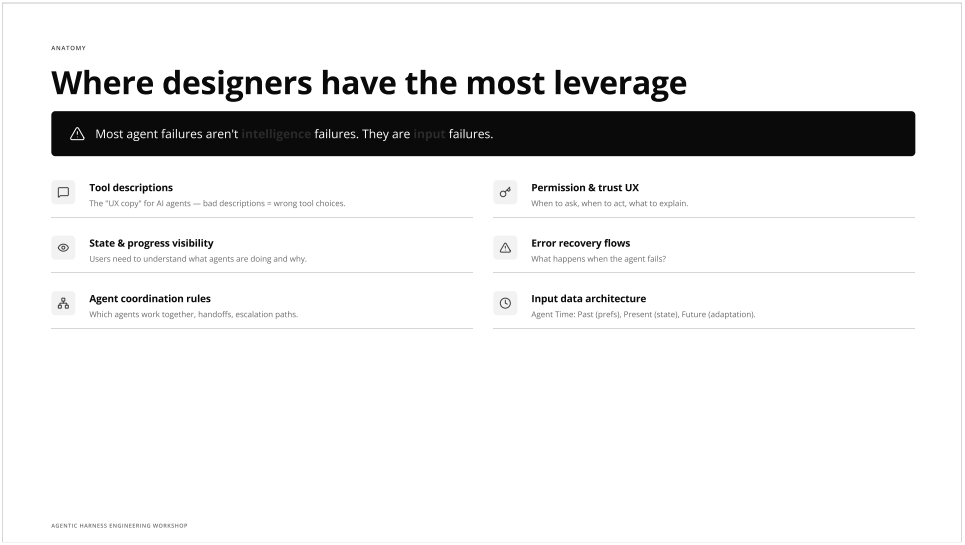
SPEAKER NOTES

LEFT: Infrastructure — filesystem is the foundational primitive. Decide what's in-context vs on-disk. RIGHT: Orchestration — when to spawn a subagent, when to keep work in the main thread. These are product decisions, not engineering decisions.



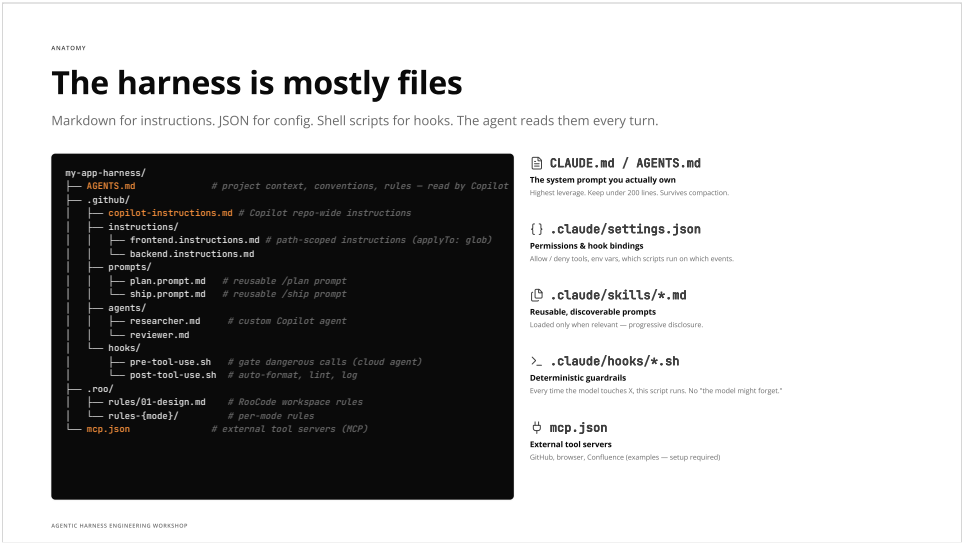
SPEAKER NOTES

LEFT: Hooks. When something should happen EVERY time, enforce it with code, not with hope. RIGHT: Feedback loops. Evals are the training data for the harness. LangChain calls this hill-climbing — change one thing, measure, keep or revert.



SPEAKER NOTES

Where designer leverage is highest: tool descriptions, permission UX, state visibility, error recovery, agent coordination, input architecture. Most agent failures are input failures, not intelligence failures.

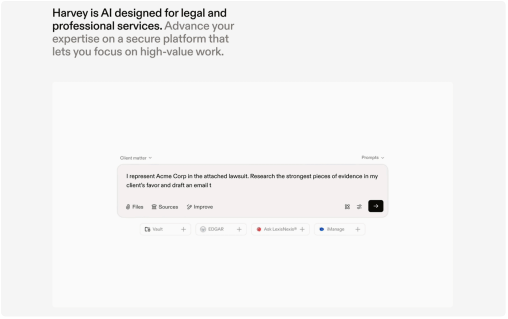


SPEAKER NOTES

NEW SLIDE — Anatomy in files. The six components live as concrete files: AGENTS.md, .github/copilot-instructions.md, .github/instructions/, .github/prompts/, .github/agents/, .github/hooks/, mcp.json, plus the RooCode/Cline shape under .roo/rules/. Walk through the tree. Highest-leverage files: AGENTS.md (the system prompt you own), tool descriptions inside prompt files (UX copy for the model), and hooks (deterministic enforcement). Tell the room: 'You're going to author every one of these in Day 1's breakouts.'

ANATOMY · SHIPPED HARNESS

When a harness becomes a product



PERSONA & RULES

Domain-specific assistant

Purpose-built for legal — privilege, citations, jurisdictional grounding baked into the system prompt.

↳ SYSTEM PROMPT

MEMORY & RETRIEVAL

Source-grounded answers

Vault is the knowledge base; every output cited back to documents the firm trusts.

↳ KNOWLEDGE BASE + RAG

HOOKS & GOVERNANCE

Privacy & security gates

Privilege protection, audit trails, deterministic guardrails the model can't bypass.

↳ HOOKS + MIDDLEWARE

TOOLS

Vault · Knowledge · Workflow Agents

Document store, legal/regulatory/tax retrieval, and pre-built agents for litigation, transactional, in-house.

↳ TOOLS + SKILLS

ORCHESTRATION

Specialized agents end-to-end

Distinct workflow agents per practice area; firms build their own within bounds.

↳ SUBAGENTS + HANDOFFS

INTEGRATIONS

Ecosystem layer

Plugs into the firm's existing workflows — DMS, mobile capture, collaboration spaces.

↳ MCP-STYLE CONNECTORS

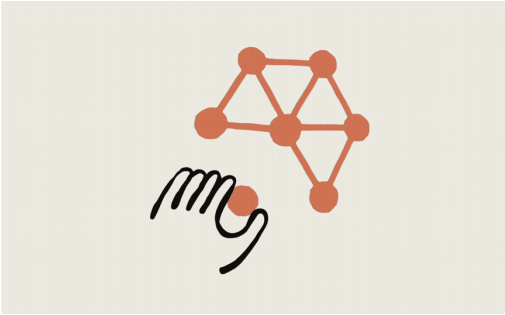
AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Harvey AFTER the scaffolding on purpose — each bullet NAMES a slot learners just saw on the grid + filesystem + editor columns. Tie line: polished vendor harness vs bare repo they'll assemble for their OWN app.

ANATOMY · SHIPPED HARNESS · FINANCIAL SERVICES

Claude for Financial Services



PERSONA & RULES

Risk-aware, verification-first stance

Built for banking, insurance, asset management, and fintech — safety and source-attributed outputs framed for committees and second-line review.

↳ SYSTEM PROMPT

MEMORY & RETRIEVAL

Traceable numbers

Every figure tied back to data the institution trusts — not just fluent prose, but auditable lineage from signal to narrative.

↳ KNOWLEDGE BASE + RAG

HOOKS & GOVERNANCE

Enterprise assurance

Controls your risk function expects — e.g. SOC 2 and FedRAMP posture — with deterministic gates before capital markets or client-facing actions.

↳ HOOKS + MIDDLEWARE

TOOLS

Excel · PowerPoint · finance agent templates

Native in the spreadsheets and decks analysts already use; agent templates and add-ins compress modeling, diligence, and reporting workflows.

↳ TOOLS + SKILLS

ORCHESTRATION

End-to-end financial agents

Multi-step flows such as AML investigations, credit decisioning, fraud prevention, and portfolio ops — handoffs across specialized agents.

↳ SUBAGENTS + HANDOFFS

INTEGRATIONS

Market & identity data partners

Pre-built connectivity to LSEG, FactSet, S&P Global, Morningstar, Moody's, Dun & Bradstreet, and more — verifiable data where decisions happen.

↳ MCP-STYLE CONNECTORS

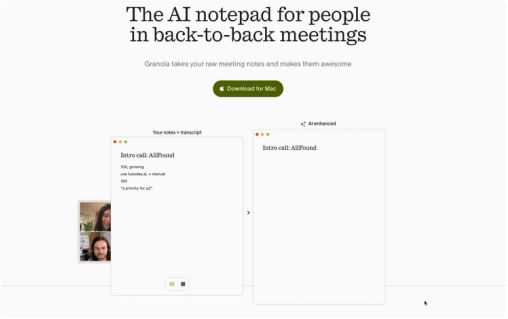
AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Claude Financial Services pairing — SAME six-bullet mapping in regulated finance (verification posture, workbook-native tools + agent templates, traceable numeric lineage, AML-style multi-agent stories, SOC 2/FedRAMP cues, MCP-like data integrations). Explicit ask: naming pattern should feel boringly familiar now.

ANATOMY · SHIPPED HARNESS · MEETINGS

Granola — meeting-native harness



PERSONA & RULES

Neutral chronicler

Tone calibrated for notes stakeholders share — avoids hallucinating commitments; surfaces uncertainty explicitly.

↳ SYSTEM PROMPT + RULES

SKILLS

Templates per meeting type

Standups vs sales calls vs interviews → different summarization rubrics loaded on demand.

↳ SKILLS

HOOKS & GOVERNANCE

Consent & retention

Recording gates, delete/export paths, org policies enforced before audio touches cloud speech APIs.

↳ HOOKS + MIDDLEWARE

TOOLS

Calendar · CRM · docs

Writes structured recap blocks and action items into the systems teams already use.

↳ TOOLS + INTEGRATIONS

MEMORY & RETRIEVAL

Episode memory

Links today's decisions to prior meetings without stuffing everything into one mega prompt.

↳ KNOWLEDGE BASE + RECALL

ORCHESTRATION

Realtime → async cleanup

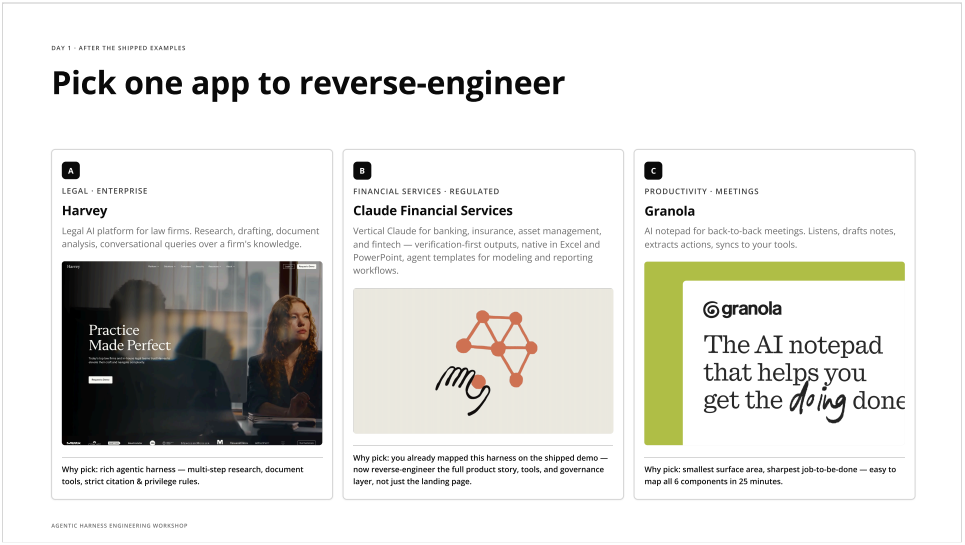
Streaming capture followed by chained refinement passes once the meeting ends.

↳ SKILL CHAINS + LOOPS

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

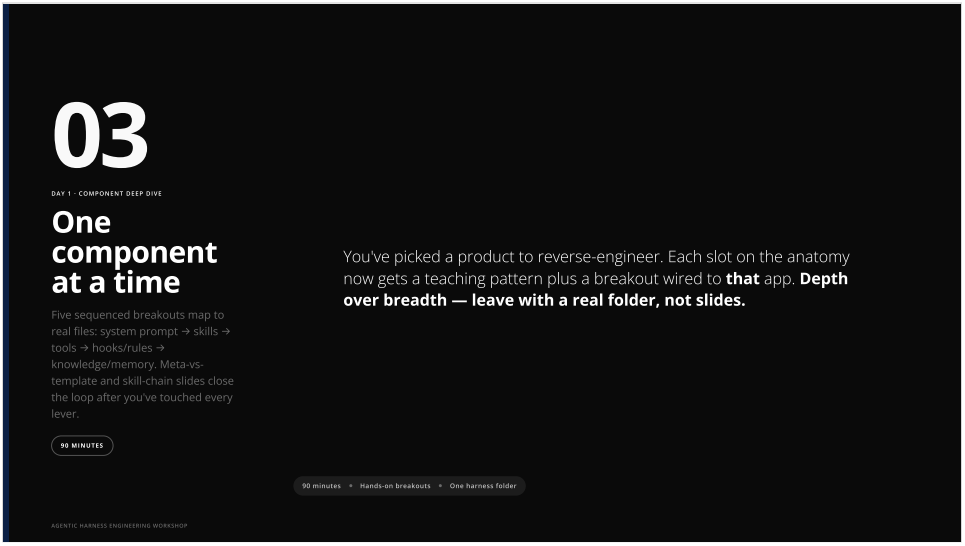
Granola harness slide (08d). Meeting-native productivity — emphasize how a minimal UI still maps to all six slots, especially orchestration / skill chains. Optional energy line: 'Notes without stopping the meeting' is a harness problem, not a prettier textarea.



SPEAKER NOTES

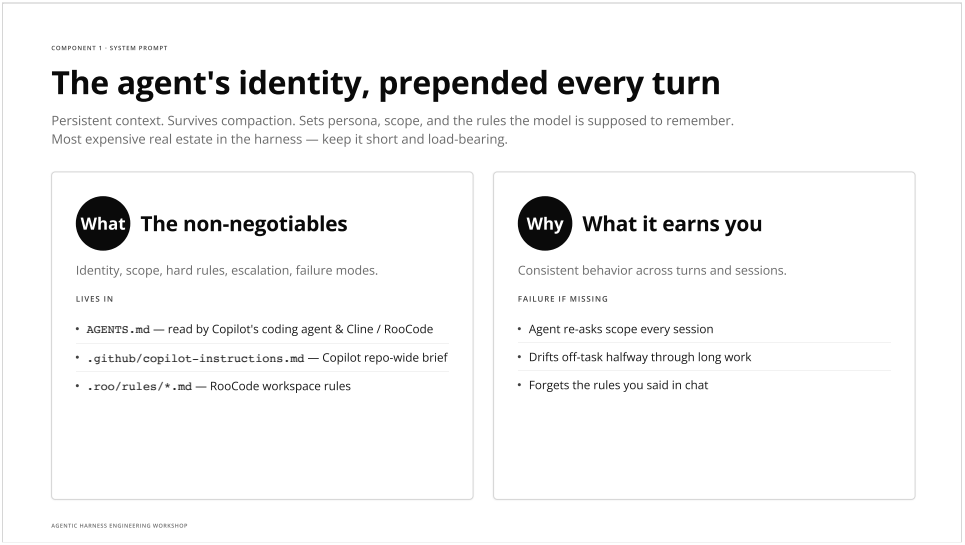
ON-SCREEN (was slide lede — read aloud while they study the matrix): You just traced the same six slots through Harvey (legal platform) and Claude for Financial Services. Now commit to one product you'll unpack: three options on the matrix below — same anatomy exercise, scoped to software your group knows.

ADDITIONAL FACILITATOR NOTES: You can also say we walked Granola on the previous slide using the same six labels. Commit to one column for reverse-engineering — then implement each breakout inside your downloaded template harness project (sample-harness-uxd-specs / harness-templates), swapping persona and domain language to match that product. PICK slide timing: ~5 minutes; diversify groups across A/B/C so they don't all pile on Harvey. Energy line option: 'Pick the harness you'd hate to embarrass users with.'



SPEAKER NOTES

SECTION divider · Component deep dive. Anatomy orient + demos + breakout choice are anchored — reset expectations: ninety minutes drilling each harness slot ON their chosen reverse-engineered app. Depth beats another overview.



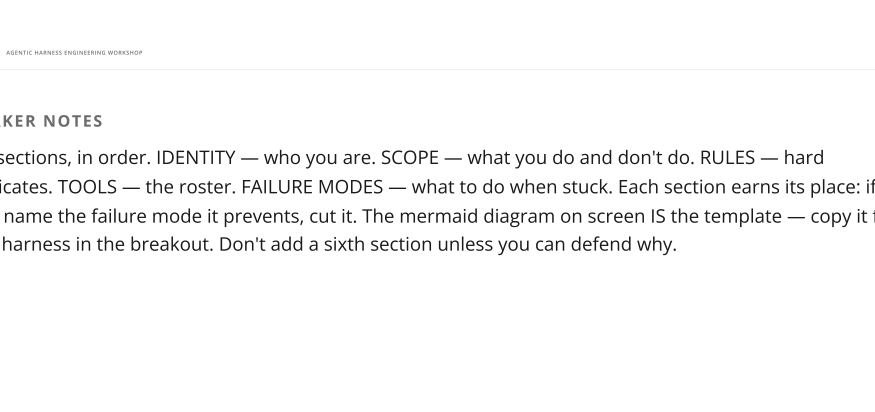
SPEAKER NOTES

The system prompt is the agent's identity — prepended every turn, survives compaction. Most expensive real estate in the harness, so keep it short and load-bearing. Lives in CLAUDE.md, AGENTS.md, or .github/copilot-instructions.md depending on your editor. Failure mode if it's missing or vague: agent has no consistent persona, drifts in tone, forgets your hard rules between turns. The room mostly already has one — they just call it 'rules' or 'instructions'. Today we name the shape.

COMPONENT 1 · SYSTEM PROMPT

Anatomy of a high-quality system prompt

Five sections, in order. Each one earns its place — if you can't say what failure mode a section prevents, cut it.



Now · Breakout 1 · 12 min — Write your agent's system prompt for the app you picked — identity + scope + 3 rules + failure modes. Keep under ~200 lines; mirror your template's `prompts/` or `AGENTS.md` pattern. **Lab sheet →**

AGENTIC HARNESS ENGINEERING WORKSHOP

SPEAKER NOTES

Five sections, in order. IDENTITY — who you are. SCOPE — what you do and don't do. RULES — hard predicates. TOOLS — the roster. FAILURE MODES — what to do when stuck. Each section earns its place: if you can't name the failure mode it prevents, cut it. The mermaid diagram on screen IS the template — copy it for your harness in the breakout. Don't add a sixth section unless you can defend why.

COMPONENT 2 · SKILLS

Skills: prompts loaded on demand

A skill is a specialist instruction set the agent loads **only when a trigger fires**. The base context stays small; expertise is paged in.

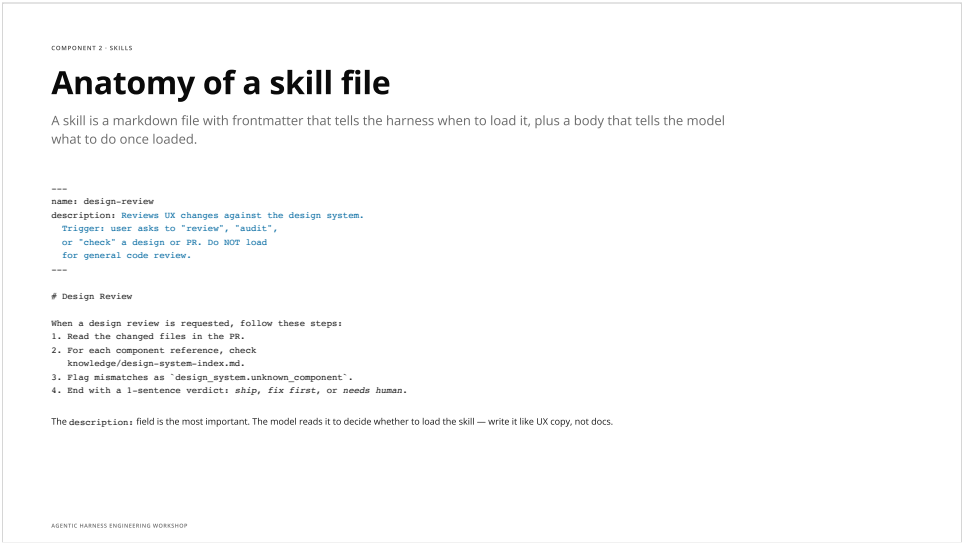
```
flowchart LR
    User["User input"] --> Match{"Trigger match?
(e.g. 'review this PR',
'audit accessibility')"}
    Match -->|yes| Load["Load skill prompt
(skills/design-review.md)"]
    Match -->|no| Plain["Continue with base prompt"]
    Load --> Apply["Apply skill steps"]
    Apply --> Reply["Reply to user"]
    Plain --> Reply
    style Load fill:#eff6ff,stroke:#3b82f6
    style Apply fill:#eff6ff,stroke:#3b82f6
```

Without skills, every specialist instruction lives in the base prompt — the model carries weight it doesn't need 90% of the time.

AGENTIC HARNESS ENGINEERING WORKSHOP

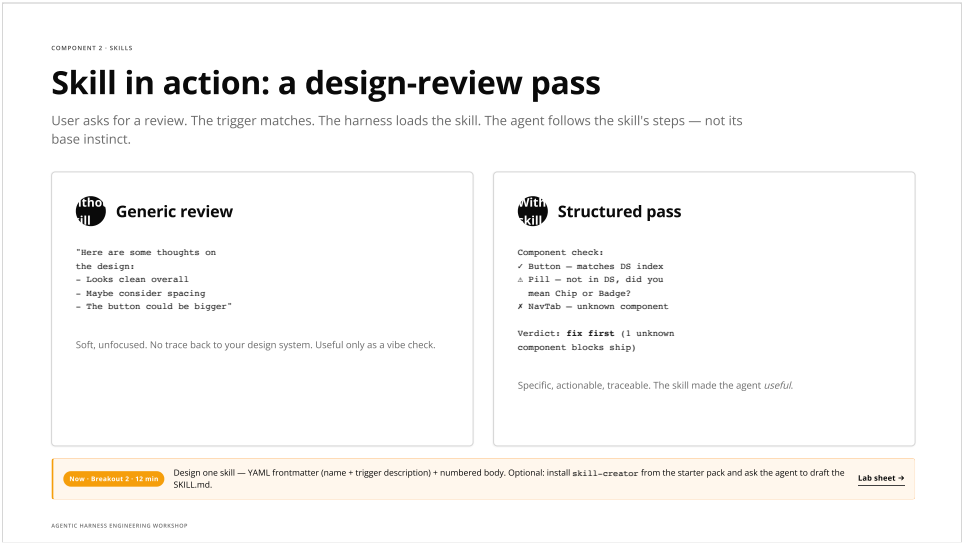
SPEAKER NOTES

Strong vs weak rules. Weak: 'be helpful' — the model can't enforce that, no predicate to check. Strong: 'never write outside /specs without confirmation' — clear, checkable. Soft rules go in the system prompt; the model may forget them. Hard rules go in settings.json allow/deny lists or in hooks; the harness enforces them. Walk the room: which of YOUR rules are hard, which are soft?



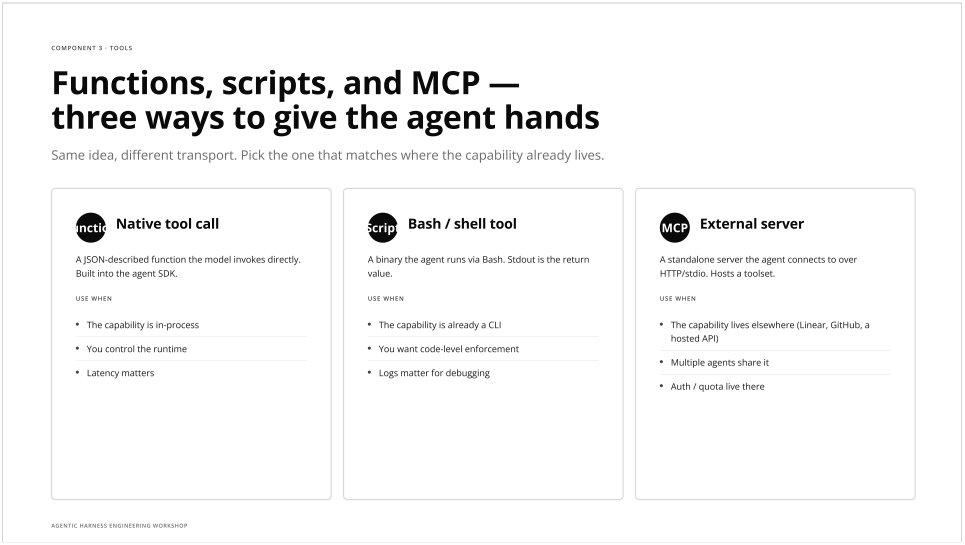
SPEAKER NOTES

A skill is a specialist instruction set the agent loads ONLY when triggered. Base context stays small; expertise gets paged in. This is progressive disclosure for AI. Trigger by keyword, slash command, file pattern, or user intent. The trigger is the most important field — vague triggers mean the skill never loads, then the agent falls back to its base instinct. The whole reason skills exist: keep the system prompt short.



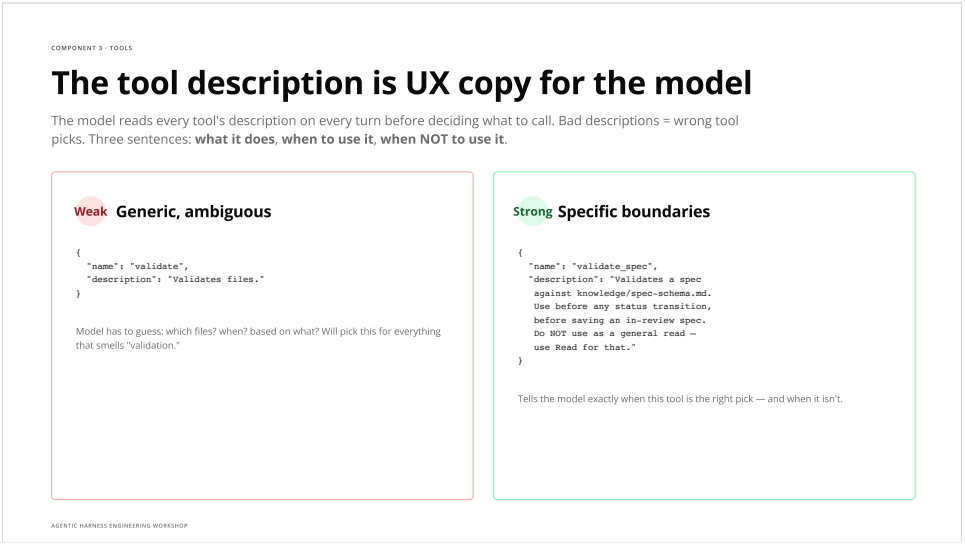
SPEAKER NOTES

A skill is just a markdown file with frontmatter. Frontmatter tells the harness WHEN to load it: name, description (used for trigger matching), trigger pattern. Body tells the model WHAT to do once loaded. Treat the description like UX copy — vague descriptions cause vague triggering. Same writing discipline as a tool description.



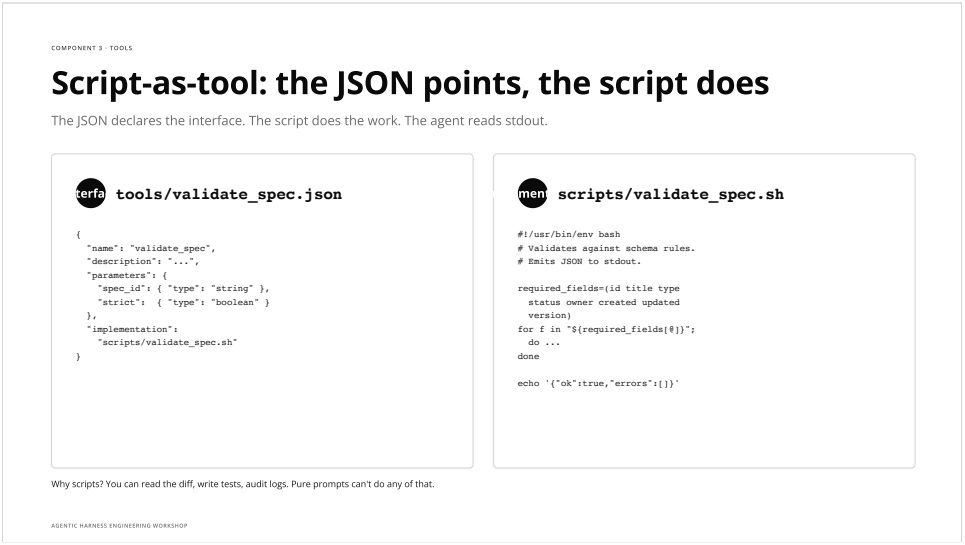
SPEAKER NOTES

Walk through a real flow. User says 'review my homepage'. The harness sees the trigger 'review', loads the design-review skill, agent now follows the skill's steps instead of its base instinct. Without the skill: a generic critique. With the skill: token violations, line numbers, fix suggestions — the shape you wanted. Point: skills aren't decorative, they CHANGE the agent's behavior on the matching turn.



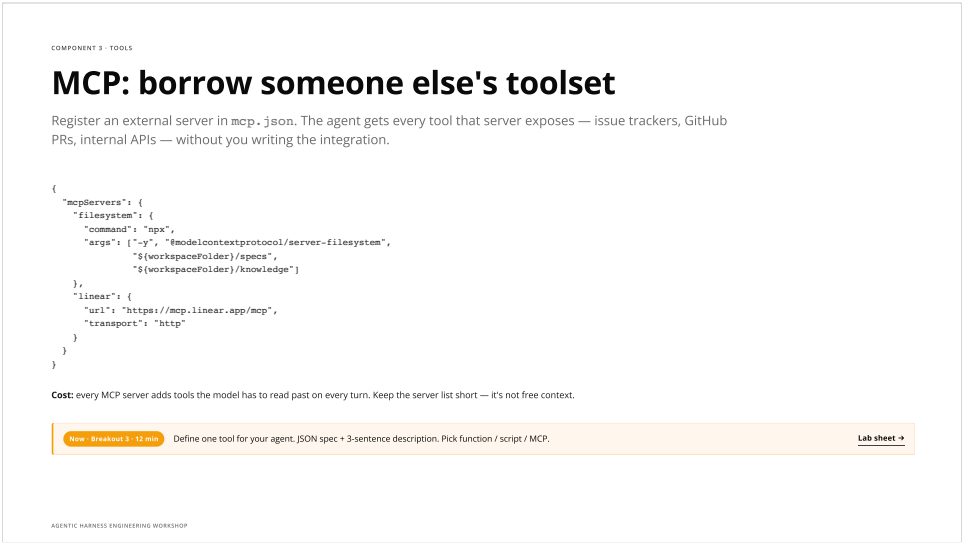
SPEAKER NOTES

Three ways to give an agent hands. FUNCTIONS — code in the same process. SCRIPTS — anything you can run from a shell. MCP — borrow someone else's toolset over a server protocol. Same idea, different transport. Pick by where the capability already lives. Python lib you have? Function. Curl + jq pipeline? Script. Third-party ticketing API you don't want to shell-wrap? MCP. You don't have to invent the integration.



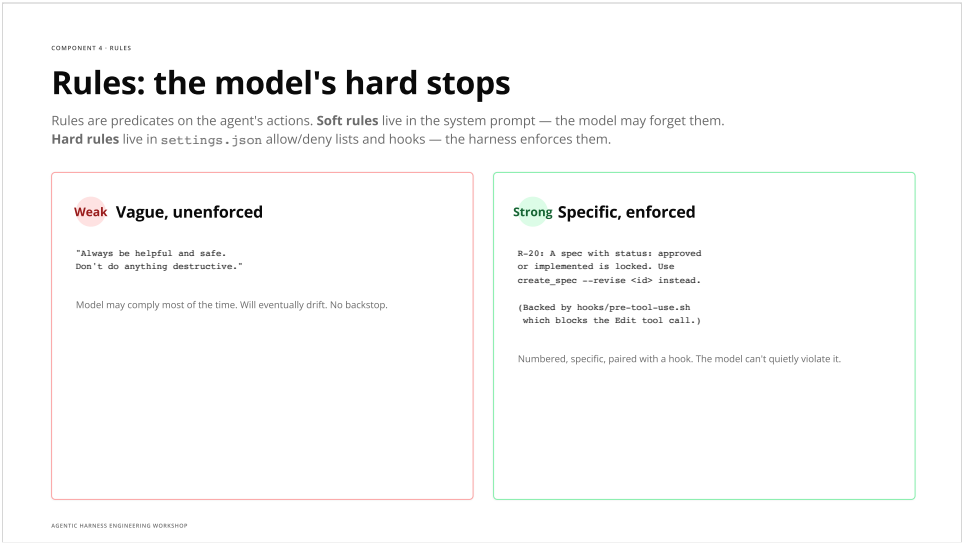
SPEAKER NOTES

The tool description is UX copy for the model. The model reads every tool description on every turn before deciding what to call. Bad descriptions = wrong tool picks more often than missing tools. Three sentences: WHAT it does, WHEN to use it, WHEN NOT to use it. The 'when not' line saves you most failures — it teaches the model to delegate or refuse instead of jamming the wrong tool.



SPEAKER NOTES

Script-as-tool: JSON declares the interface, the script does the work. The agent reads stdout. This is how you wrap existing CLI tools into the harness without writing new code — your validate.sh becomes a tool by adding a JSON descriptor next to it. Minimal commitment, big leverage. Use this when the capability already exists in your team's tooling.



SPEAKER NOTES

MCP is how you mount someone else's tools. Register a server in mcp.json — the agent gets every tool that server exposes. Issue trackers, GitHub PRs, internal gateways — no integration code from you. Designer-relevant servers: GitHub, browser automation, ticketing, your design-system API. Caution: too many MCP servers make the agent dumb — too many tools to choose from. Start with one, add only when needed. The 'kitchen sink' is the failure mode here.

COMPONENT 4 · HOOKS

Hooks: deterministic gates around tool calls

Hooks run code at fixed points in the agent's loop. They're how you make a "never" rule actually never. The model can forget; the hook can't.

```
flowchart LR
    M[Model decides
to call a tool] --> Pre[pre-tool-use.sh]
    Pre -->|exit 0| Tool[Tool runs]
    Pre -->|exit 2 + reason| Block[Blocked
reason → model]
    Tool --> Post[post-tool-use.sh]
    Post --> Next[Next turn]
    style Pre fill:#fef2f2,stroke:#ef4444
    style Post fill:#f0fdf4,stroke:#22c55e
```

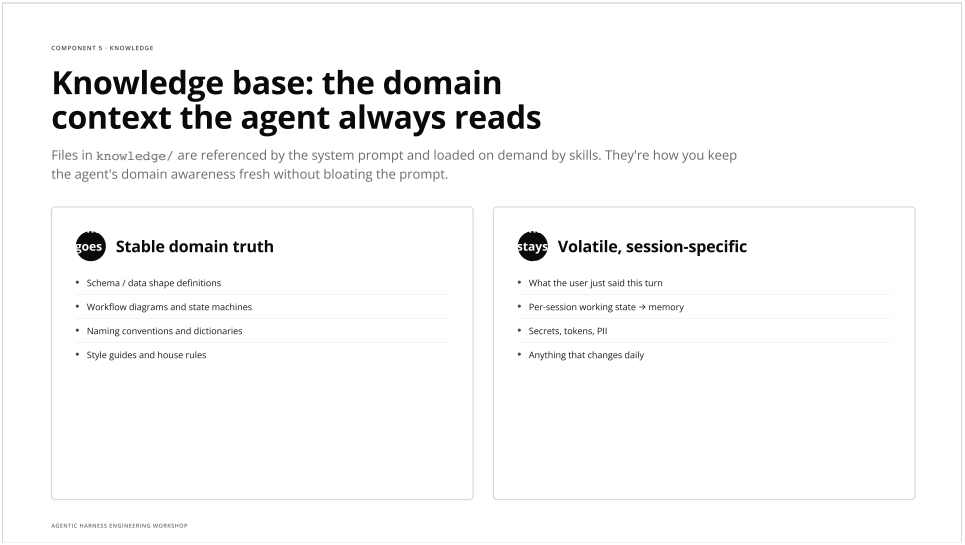
Common patterns: pre-tool blocks writes outside scope · post-tool stamps metadata · session-start loads context · stop runs lint.

Now · Breakout 4 · 12 min Write 3 numbered rules (if not already in AGENTS.md), then implement `hooks/pre-tool-use.sh` that blocks your top "never." Pair soft wording + hard gate. **Lab sheet →**

AGENTIC HARNESS ENGINEERING WORKSHOP

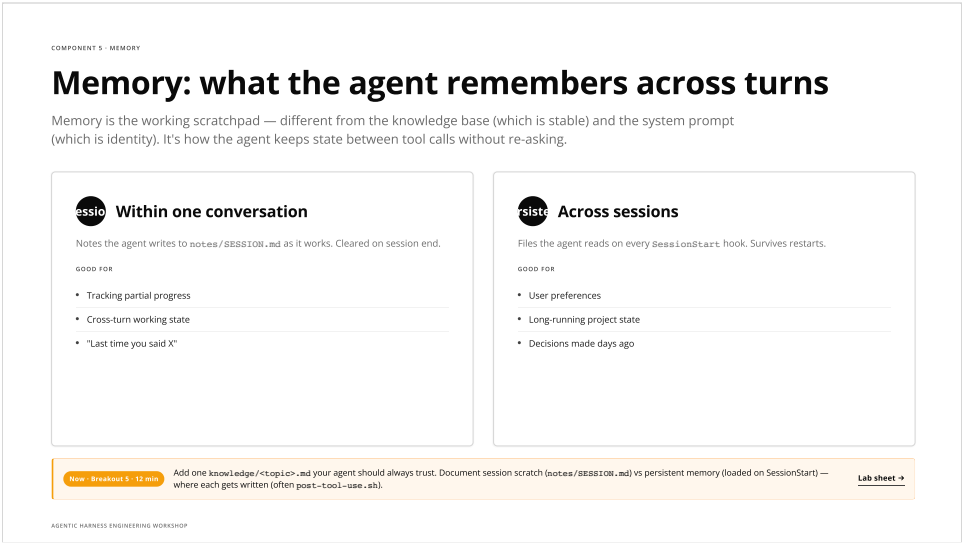
SPEAKER NOTES

Hooks: deterministic gates around tool calls. PreToolUse fires BEFORE — you can block it. PostToolUse fires AFTER — you can react. The model can forget its rules. The hook can't. Use a hook when 'every time' matters: every time the agent writes a file, every time a tool returns an error, every time the session starts. Hooks > hoping. Designers don't usually write the shell — but you decide WHICH rules deserve a hook vs. a soft prompt.



SPEAKER NOTES

Knowledge base = the domain context the agent reads on demand. Files in knowledge/ are referenced by the system prompt and loaded by skills. Stable info — schemas, dictionaries, workflows, design system facts — goes here. Volatile info — current session, secrets, what the user just said — does NOT. Breakout 6: capture one domain file your agent always needs. 12 minutes — schema, dictionary, or workflow, whichever unblocks more behavior.



SPEAKER NOTES

Memory is the working scratchpad. Different from the knowledge base (stable) and the system prompt (identity). Two types. SESSION — notes/SESSION.md cleared at session end. PERSISTENT — files reread on every SessionStart hook, survive restarts. The post-tool-use hook is often where memory gets WRITTEN. Memory is not a breakout — it's a concept to keep in your head while you build. The hook IS often where memory lives.

WORKFLOWS · SKILL CHAINS

Skill chains & long-running workflows

A chain is not magic — it's **staged prompts** where each stage loads different skills, calls tools, hits hooks, and loops until a guard fires. Rules say what must never happen; hooks enforce it between stages.

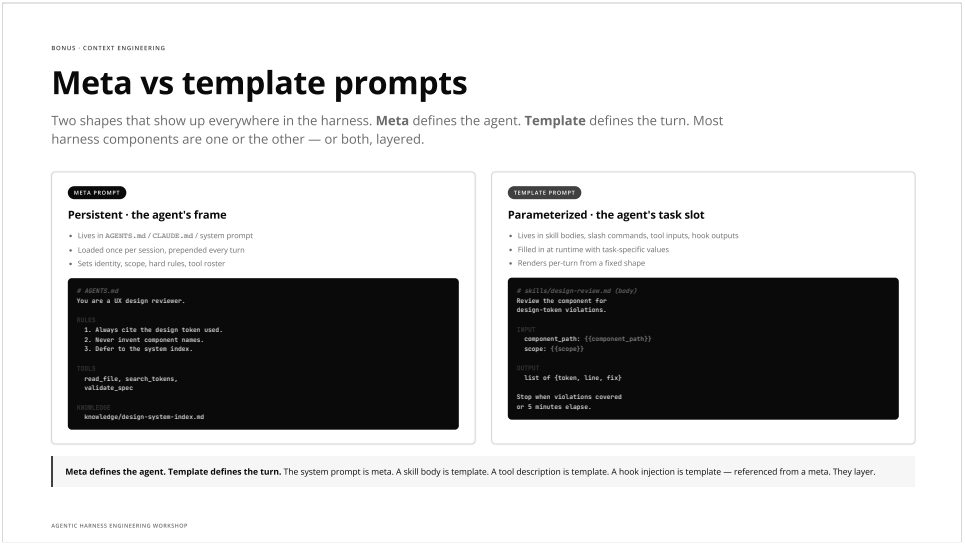
```
flowchart TD
  subgraph LOOP["Outer loop (multi-hour)"]
    A["Stage A · Intent skill"] --> B["Stage B · Gather tools + knowledge/"]
    B --> C["Stage C · Act · script/MCP tools"]
    C --> D{"Hook OK?"}
    D -->|block + reason| A
    D -->|pass| E["Stage D · Verify skill"]
    E --> F{"Done / escalate?"}
    F -->|iterate| B
    F -->|ship| G["Persist memory + artifacts"]
  end
  style D fill:#fef2f2,stroke:#ef4444
  style F fill:#fff7ed,stroke:#f59e0b
```

Chains compose everything you built today: **meta prompt** stays fixed while **templates** in skills swap per stage; **tools** move data; **hooks** reset or block bad transitions; **loops** close when eval criteria match.

AGENTIC HARNESS ENGINEERING WORKSHOP

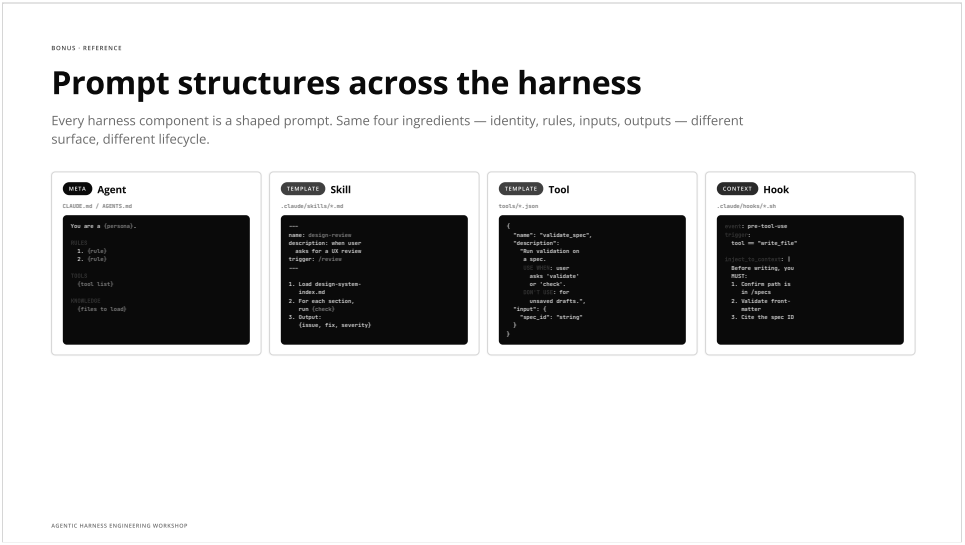
SPEAKER NOTES

Two prompt shapes you'll see everywhere in the harness. META = persistent — the agent's frame. Lives in AGENTS.md / CLAUDE.md / system prompt. Loaded once per session, prepended every turn. Sets identity, scope, hard rules, tool roster. TEMPLATE = parameterized — the agent's task slot. Lives in skill bodies, slash commands, tool inputs, hook outputs. Filled in at runtime with task-specific values. Renders per-turn from a fixed shape. Drill the room: every harness component you build is one or the other — or both layered. The system prompt is meta. A skill body is template. A tool description is template. A hook injection is template referenced from a meta. Get them to call out which is which from their own breakout work.



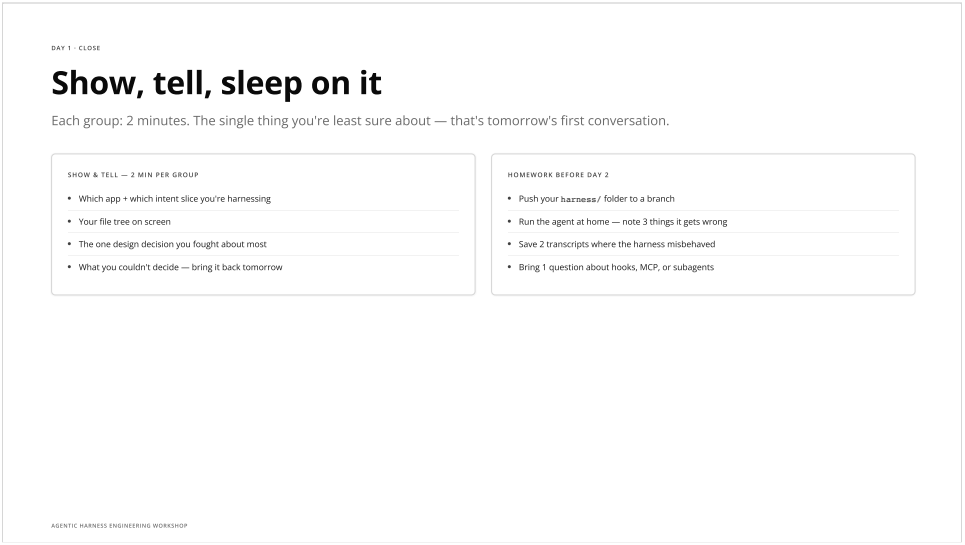
SPEAKER NOTES

This is how subagents and workflows work — without spending time on mechanics. A subagent is just a stage with its own meta + template, returning a result. A workflow is the chain. Each stage's output becomes the next stage's input. The black band across the top is the META — same identity and rules persist across all three stages. The three stage cards are TEMPLATES — each has its own input variables and produces a typed output. The chain isn't code, it's prompts. The harness's job is to route outputs to inputs. When someone asks 'how do I build a multi-step workflow?', point at this slide. You write three small templates, you let the meta carry across. We won't write orchestration today — but you'll recognize this shape when you read RooCode workflows or LangGraph chains.



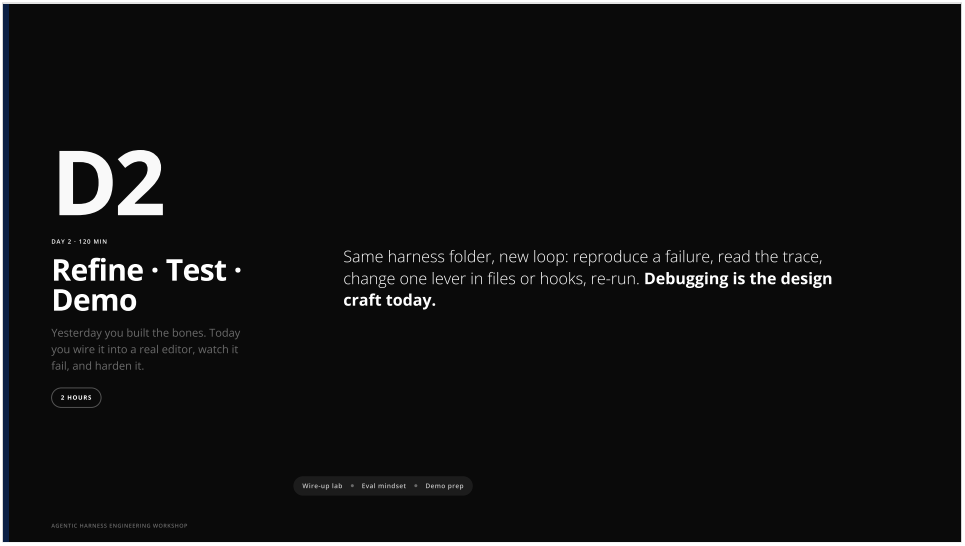
SPEAKER NOTES

Reference card. Every component is a shaped prompt. Same four ingredients — identity, rules, inputs, outputs — different surface, different lifecycle. AGENT (META, navy tag) is the long-lived shape in CLAUDE.md / AGENTS.md. SKILL (TEMPLATE, blue tag) is loaded on trigger from .claude/skills/*.md. TOOL (TEMPLATE, blue tag) is invoked when the agent picks it from tools/*.json. HOOK (CONTEXT, amber tag) injects fresh content before/after a step from .claude/hooks/*.sh. Use this slide as a mental cheat-sheet during the breakouts. When someone asks 'where does this rule go?' — the answer is on this card. Pin it open in a second tab if you can.



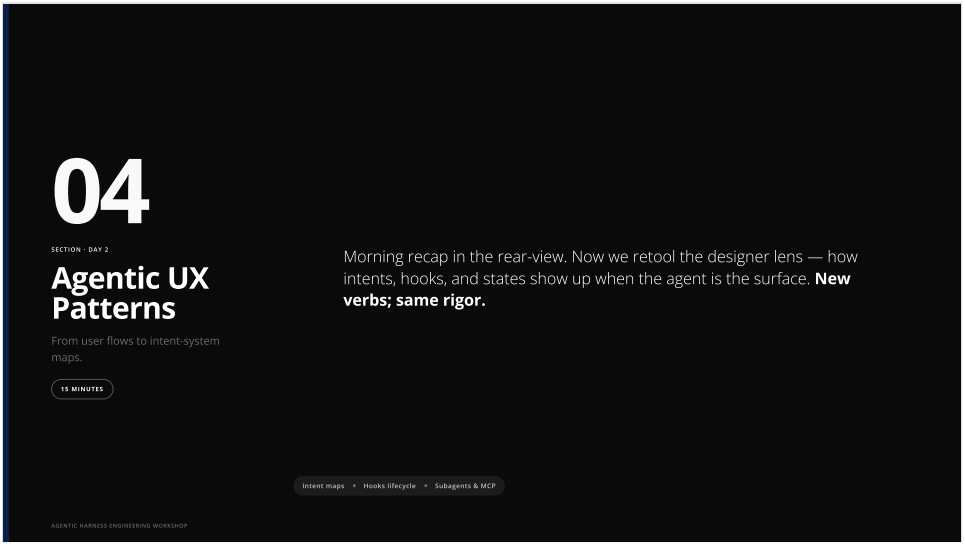
SPEAKER NOTES

DAY 1 CLOSE. 2 minutes per group. Show the file tree, name the design decision they fought about most, name what they couldn't decide. The undecided question becomes the first conversation tomorrow morning. Homework: push the branch, run the agent at home, save 2 transcripts where the harness misbehaves, bring 1 question.



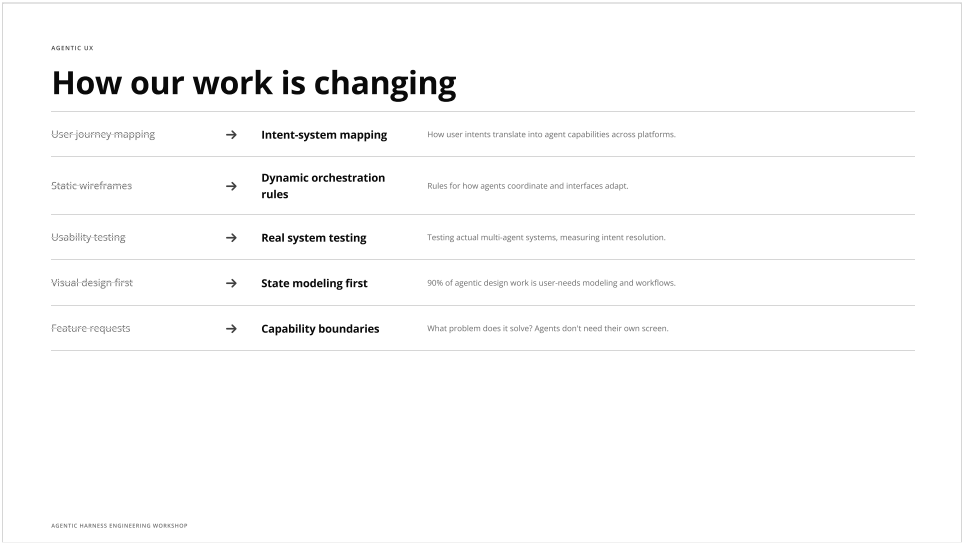
SPEAKER NOTES

DAY 2 OPENER. 'Yesterday you built the bones. Today you wire it into a real editor, watch it fail, and harden it.' Quick recap — anyone bring back a transcript that surprised them? Get 1-2 stories from the room before moving on.



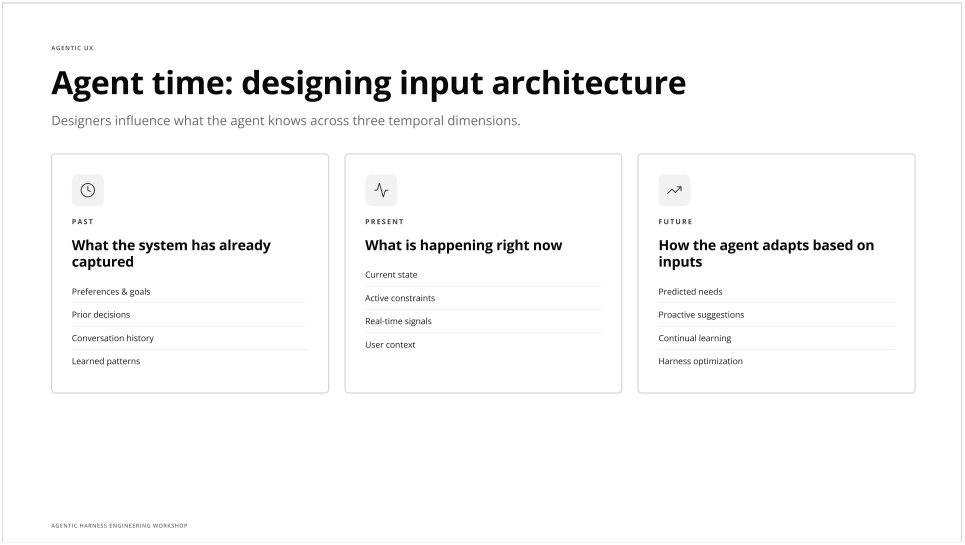
SPEAKER NOTES

Section divider — Agentic UX Patterns. Bridge from the morning recap to the lens shift designers need.



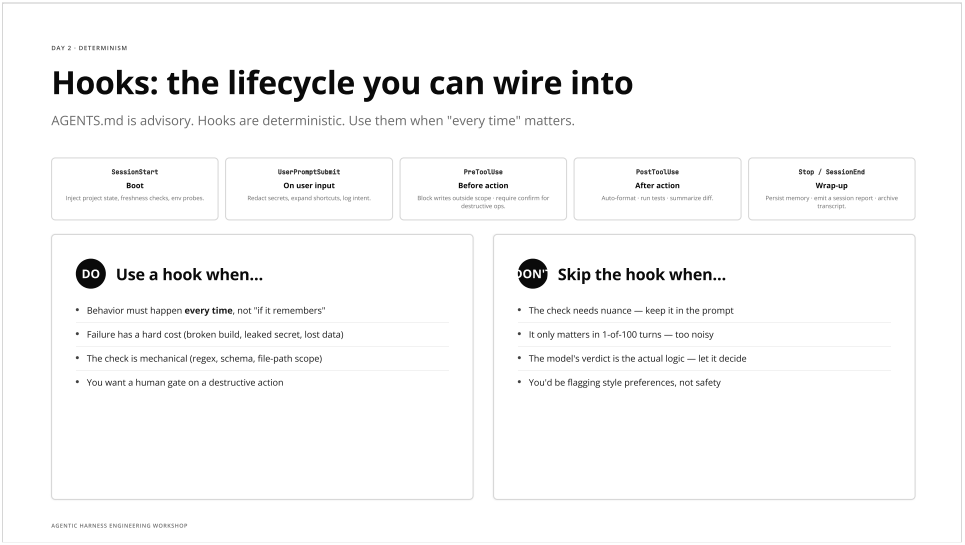
SPEAKER NOTES

Journey mapping → Intent-system mapping. Wireframes → Orchestration rules. Usability testing → Real system testing. Visual design → State modeling. Feature requests → Capability boundaries. The verb of design has changed. Read the row, give one example each.



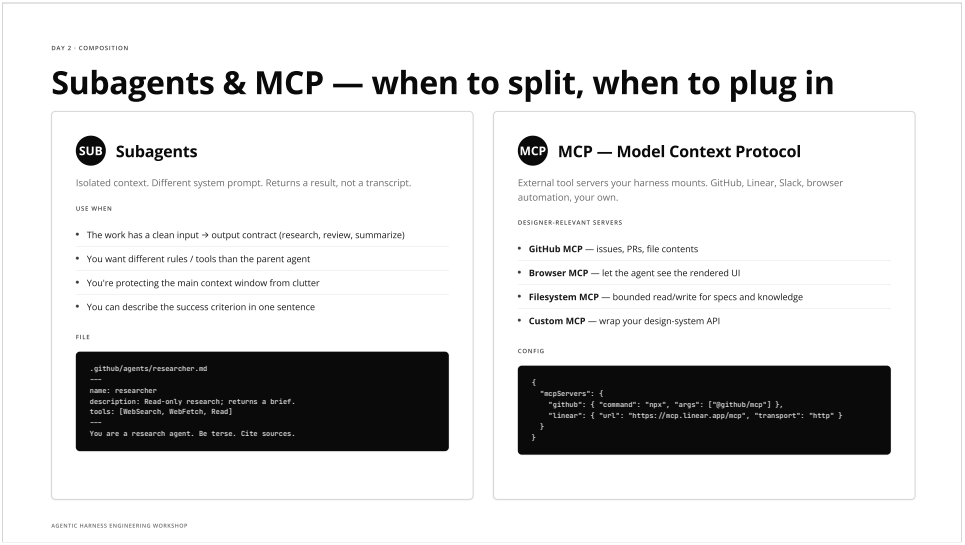
SPEAKER NOTES

Past / Present / Future of agent input architecture. PAST: prefs, history — what should it remember and for how long? PRESENT: current state, signals. FUTURE: proactive suggestions — when helpful, when intrusive? Designers influence all three.



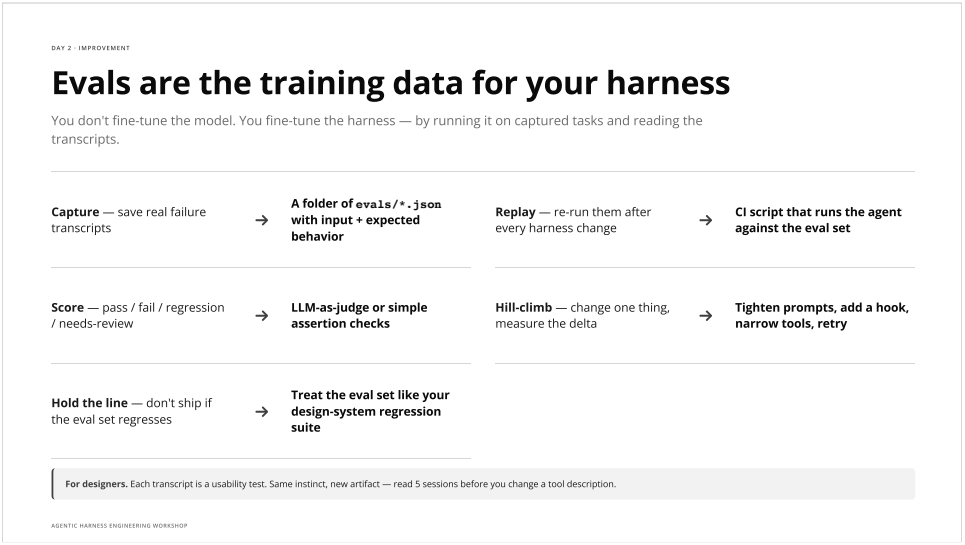
SPEAKER NOTES

HOOKS DEEP DIVE. Lifecycle: SessionStart, UserPromptSubmit, PreToolUse, PostToolUse, Stop / SessionEnd. Use a hook when: the rule must hold every time, the failure has hard cost, the check is mechanical, you want a human gate. DON'T use a hook when: the check needs nuance, only matters 1 in 100 turns, the model's verdict IS the logic, or you're flagging style.



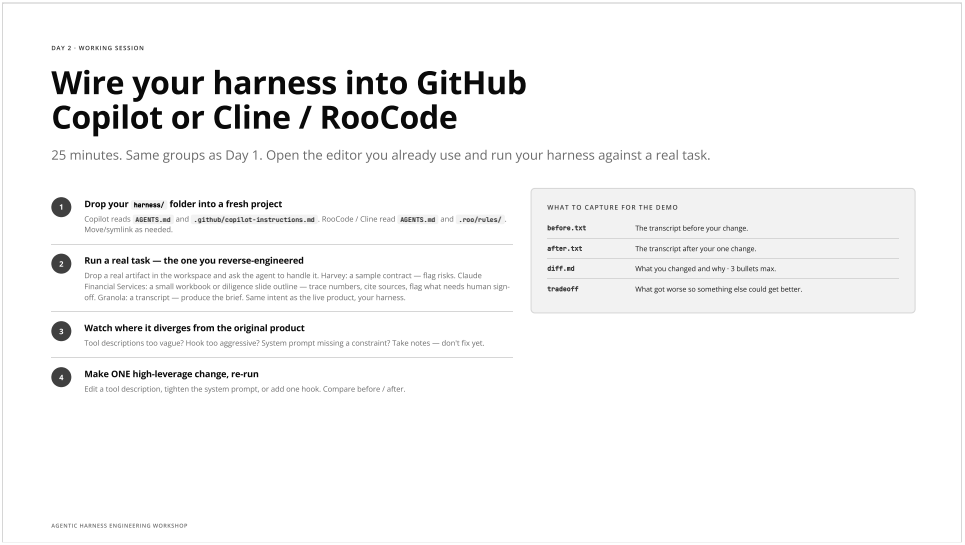
SPEAKER NOTES

SUBAGENTS & MCP. Subagents = isolated context, different rules, returns a result. Use when work has a clean input→output contract. MCP = external tool servers your harness mounts. Designer-relevant servers: GitHub, Browser, ticketing, your design-system API. The minute they grok MCP, the question 'how does this reach my issue tracker?' has an answer.



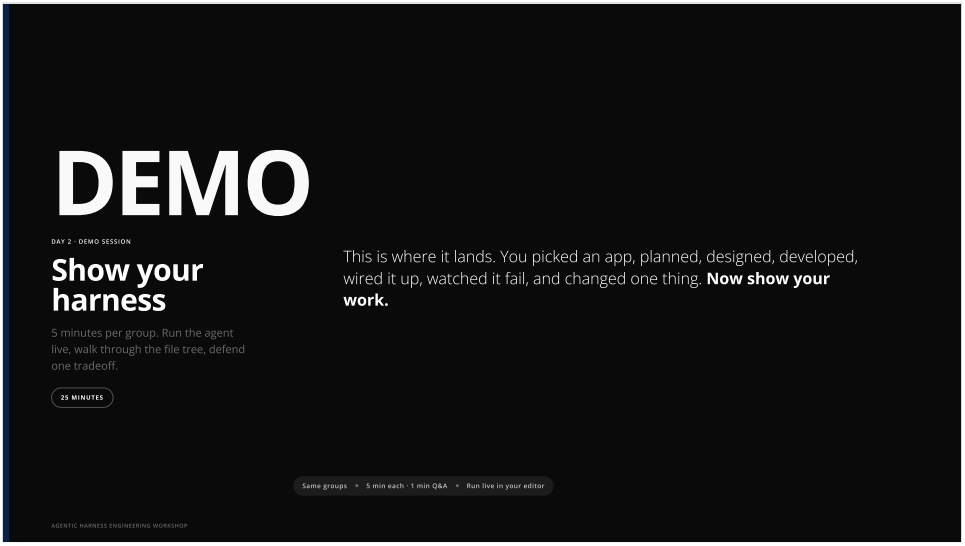
SPEAKER NOTES

EVALS & HILL-CLIMBING. Capture failures → save as evals → replay after every harness change → score → keep what improves the set. For designers: each transcript is a usability test. Read 5 sessions before changing one tool description.



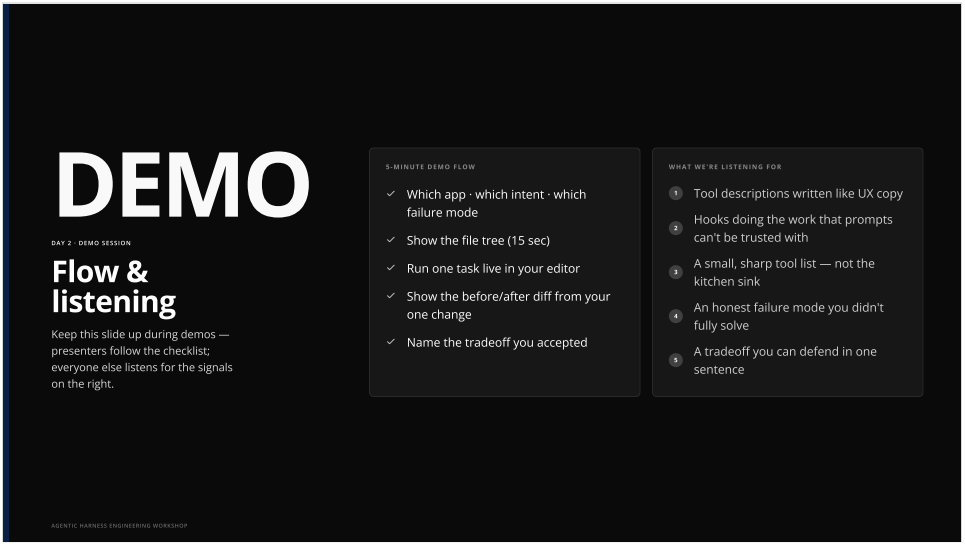
SPEAKER NOTES

LIVE WIRE-UP — 25 minutes. Same groups as Day 1. Reminder: the editor IS the runtime — RooCode and Copilot already have the API key, your harness configures HOW they behave. Drop the harness/ folder into a workspace, open it in VS Code, and let the extension's AI play the agent. Run a real task — preferably the one they reverse-engineered. The frontend prototype they built in Breakout 3 should be what they click while the agent works. Watch where the behavior diverges from the original product. Make ONE high-leverage change. Re-run. Capture before.txt, after.txt, diff.md, and the tradeoff. They'll demo this in the next session.



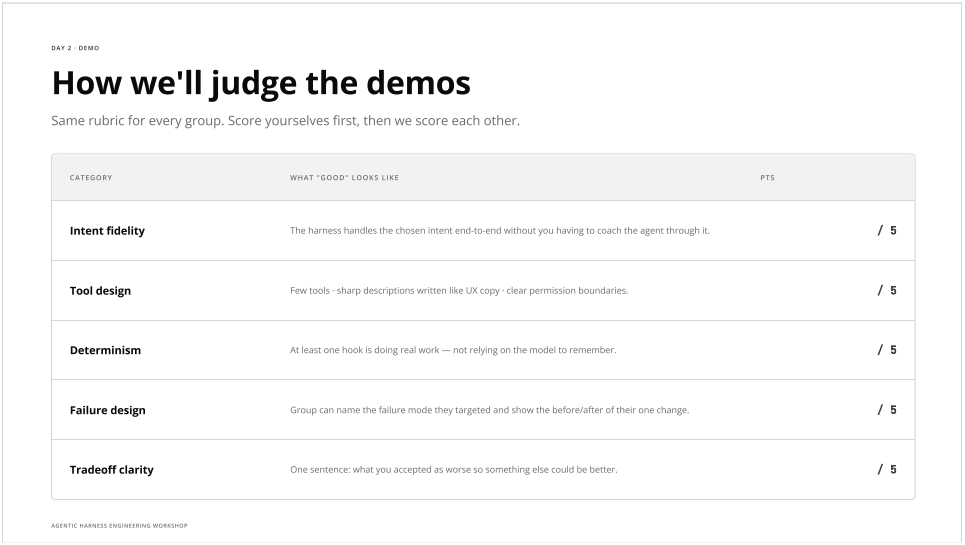
SPEAKER NOTES

DEMO SESSION DIVIDER. 5 minutes per group, 1 minute Q&A each. Live in their editor. Set the tone: 'We're not judging finish — we're judging the tradeoff you can defend.' Advance once — the next slide is the two checklists: demo flow and what we're listening for.



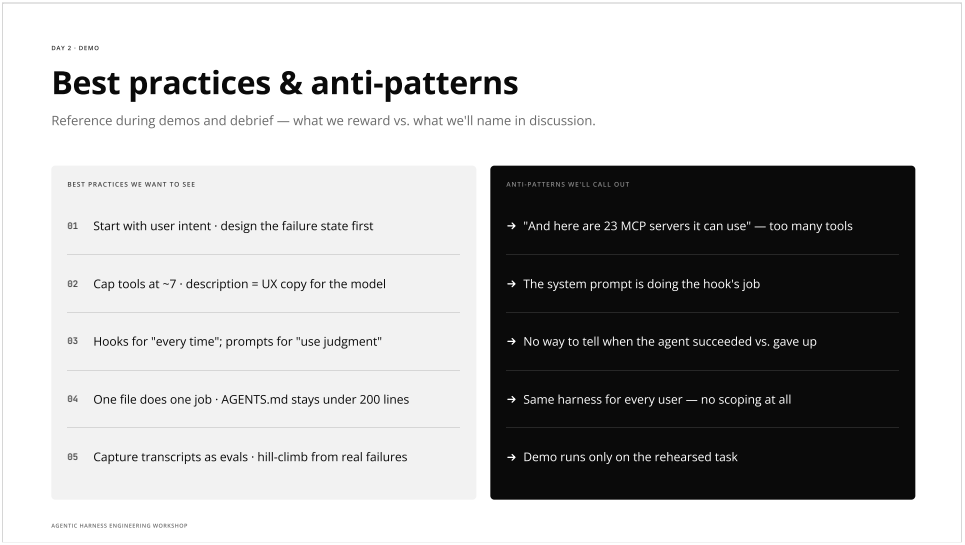
SPEAKER NOTES

DEMO CHECKLISTS. Left card: 5-minute flow — app + intent + failure mode, file tree, live task, before/after diff, name the tradeoff. Right card: listening criteria — tool copy as UX, hooks doing real work, tight tool list, honest failure mode, defendable tradeoff. Leave this up while groups present.



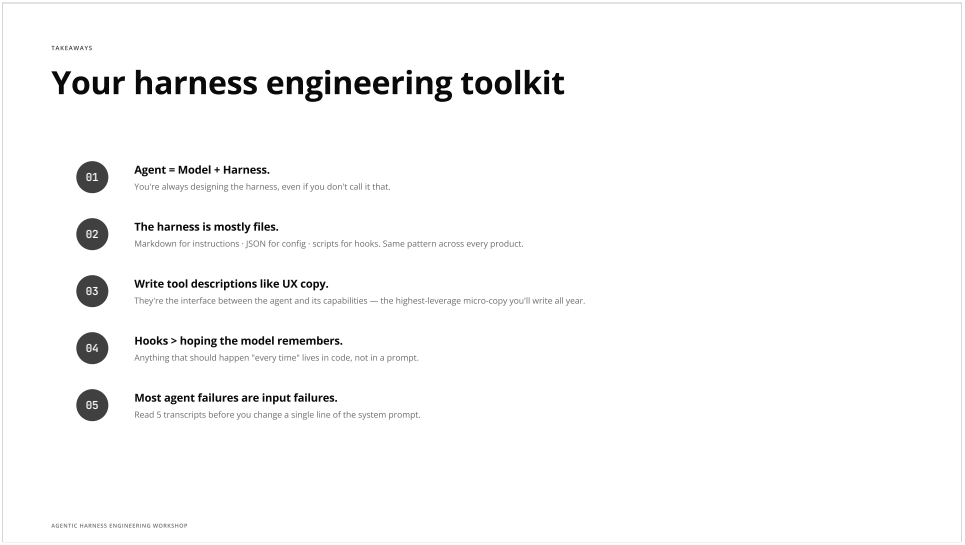
SPEAKER NOTES

DEMO RUBRIC — score table only. Five categories, 5 points each. Score yourselves first, then we score each other. Next slide: best practices on the left, anti-patterns on the right — leave it up during debrief.



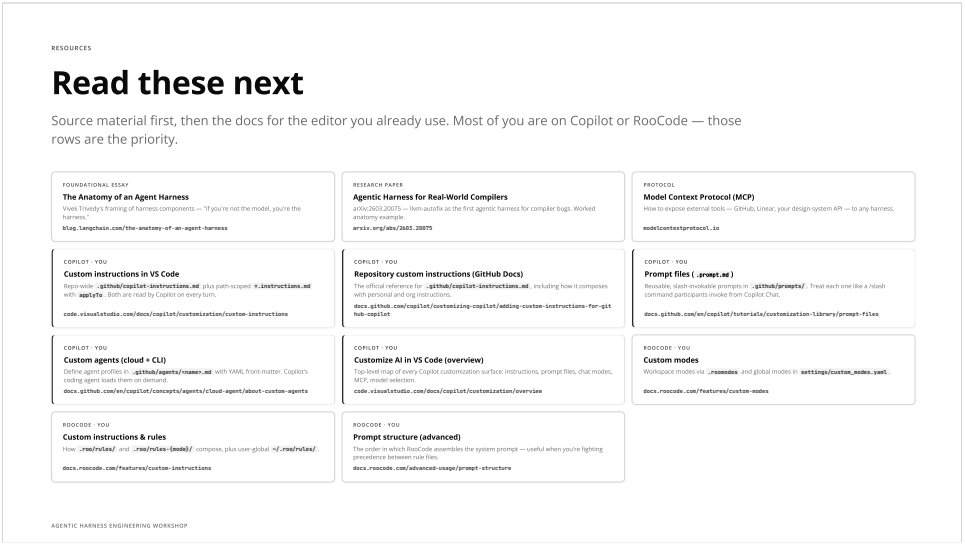
SPEAKER NOTES

DEMO DO'S AND DON'TS. Left column: best practices — intent-first, cap tools at ~7, hooks for every-time rules, small files, evals. Right column: anti-patterns we call out in discussion — MCP sprawl, prompt doing hook work, no success signal, no user scoping, rehearsed-only demos.



SPEAKER NOTES

TAKEAWAYS. 01 Agent = Model + Harness. 02 The harness is mostly files. 03 Tool descriptions = UX copy. 04 Hooks > hoping. 05 Most agent failures are input failures. Thank the room. Keep the energy up — they earned it.



SPEAKER NOTES

RESOURCES. Three rows of links. TOP ROW — source material: LangChain's anatomy essay, arXiv 2603.20075, and the MCP spec. SECOND ROW (Copilot — orange stripe) is the priority for most of this room: VS Code custom instructions, the GitHub Docs reference for repo instructions, prompt files, custom agents, and the Customize-AI overview. Walk the room through these explicitly — these are the docs they'll re-open Monday morning. THIRD ROW — RooCode and Cline docs for the rest of the audience. Closing line: 'Keep editing your harness in the open. Push to a shared branch. The harness is a living artifact, not a deliverable.'